CANNBot 畅享算子开发新旅程

CANN技术专家 邓春生

上期回顾

【彩蛋】CANNBot Agent

CANN算子优化专家: 金炯

2026.2.12

CANNBot Agent架构

Agent交互平台
负责交互/规划/执行

LLM
负责智能理解, 用户可配置

CANNBot领域知识

SKILLS

- ascendc-docs-search — 高效的文档检索
- operator-precision-debug — 算子精度调试技巧
- operator-performance-optimization — 算子性能调优

WorkFlows

- ascendc-kernel-develop-workflow — kernel直调开发流程
- operator-develop-workflow — 算子开发流程
- operator-auto-generate — 算子自动生成

Docs/Tools

- ascendc-api-docs — Ascendc基础API文档
- msprof-docs — 性能工具使用文档
- env-install-docs — 环境安装文档

演进计划和路标

本次一站式平台将预装CANNBot尝鲜版，已装备文档搜索、kernel直调开发流程等技能。

Q1上线SKILL

- **单元测试** — 单元测试生成与执行
- **集成测试** — 自动化集成测试
- **资料开发** — 文档自动生成与优化
- **代码检视** — 代码检视与修复

Q2~Q3上线SKILL

- **精度调试** — 精度诊断 | 渐进调试 tensor打印 | 仿真调试
- **性能调优** — 性能验证 | 流水分析 理论建模 | tiling设计
- **复杂算子生成** — 复杂任务规划 功能 | 性能模板拆分
- 更多编程语言, 如pypto 更多平台接入 面向下一代芯片的技巧

敬请期待：CANNBot让算子开发更简单！

CANNBot是什么？

CANNBot是基于AI Agent构建用于提升CANN开发效率、繁荣CANN生态的一系列智能体，CANNBot可兼容各类大模型和Agent框架。

核心目标

开发流程提效
结合Agent针对开发各活动效率提升，如UT/ST验证等

算子生成
实现各类语言(ASC/pyPTO等)算子端到端生成能力

模型迁移
基于昇腾CANN能力快速适配到新模型上

单算子/模型性能优化
逼近手动优化，极致性能

解决痛点

模型精度问题定位难
导入错误码诊断、常见精度问题定位skill，能自动对比排查问题

单算子调试时间长
通过skill能力完善，数周工作缩短至数天，效率倍增

性能调优能力要求高
内置智能引擎和性能优化经验库，自动探索最优策略

CANNBot三层架构，实现了从顶层业务流程到底层能力的解耦和协同

TEAMS (应用编排层)

ascendc-kernel-dev-team …

Architect
架构师

Developer
开发者

Reviewer
审核员

Tester
测试员

知识库类
- npu-arch
- tiling-design
- api-best-practices
- docs-search

工程模板类
- custom-op-to-kernel-launch

调试与测试类
- precision-debug
- runtime-debug
- op-profiling (规划中)

测试开发类
- st-design
- ut-develop
- code-review
- whitebox-design

TEAMS (应用编排层)

定义“做什么”，负责顶层业务流程的编排。当前核心实现为 ascendc-kernel-dev-team。

AGENTS (角色执行层)

定义“谁来做”，扮演特定角色执行任务。核心角色包括 architect, developer, reviewer, tester。

SKILLS (知识能力层)

定义“怎么做”，提供原子化的知识和能力。包含四大类共14个技能，如知识库类、工程模板类、调试与测试类等。

通过Teams 编排 Agents，Agents 组合 Skills系列设计，重点为开发者解决如下问题：
1、针对上下文占用大容易出现幻觉问题，通过多Agent解决。
2、知识能力在不同应用间可复用，功能更聚焦。

丰富的Skills技能库，构建CANNBot全面的能力体系

Skill类别	Skill名称	功能
知识库类	ascendc-api-best-practices	API 使用最佳实践、参数限制
	ascendc-npu-arch	NPU 架构知识、芯片型号映射
	ascendc-tiling-design	Tiling 设计方法论，按算子类别分类
	ascendc-docs-search	API 文档索引 + 在线搜索
工具辅助类	ascendc-env-check	NPU 设备查询、CANN 环境验证
	ascendc-task-focus	任务聚焦，解决长任务"迷失在中间"问题
工程模板类	custom-op-to-kernel-launch	自定义算子开发工程生成
调试与测试类	ascendc-precision-debug	精度调试，症状-原因速查、常见陷阱
	ascendc-runtime-debug	运行时错误调试，错误码解析、Kernel 挂起排查
开发自测试类	ascendc-ut-develop	UT 开发与覆盖率增强
	ascendc-st-design	aclnn 接口测试用例设计、L0/L1 用例生成
	ascendc-code-review	代码检视方法论、5 大类别规范

CANNBot 部分应用案例

智能辅助设计

分钟级需求转设计
交付标准化

深度理解需求，自动生成高质量设计文档，覆盖关键设计点，避免人为疏忽。

UT用例智能生成

0到80%覆盖率跃升
用例可验证

自动分析代码逻辑，生成全面的单元测试用例，附带详细断言，提升测试效率。

深度代码Review

精准识别编码红线
消除深层缺陷

深入代码内部，识别空指针、资源未释放等深层问题，消除潜在Bug和技术债。

ST用例分层设计

多维因子组合
保障完整性与深度

基于测试因子、约束和依赖关系智能组合，生成覆盖接口、业务流程的全面用例。

AI驱动算子生成

端到端自动化开发
全维度达标交付

接收需求描述，自动执行设计、编码、自测试流程，交付功能、精度、性能达标的算子。

模型调优

自动针对昇腾 NPU 推理开展优化并验证

积累了在真实模型针对原子优化项进行组合运用。把怎么选、按什么顺序组合、每步怎么验证的路径知识模块化

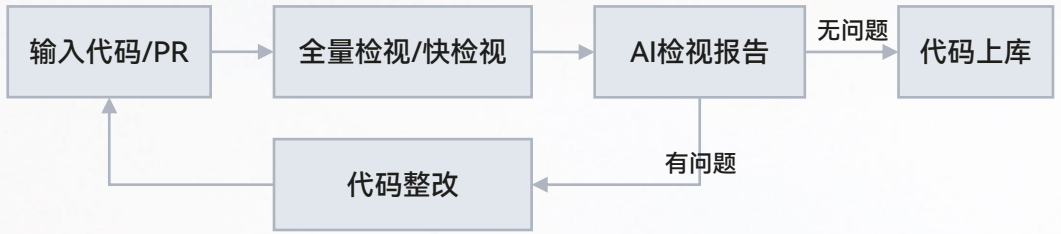
CANNBot应用：代码Review

支持本地文件、PR合入两种检视模式，支持5类典型编码规范(数值运算、内存指针、资源管理、输入验证、并发安全）检视，编码红线问题拦截率达到90%+，检视效率从天级别提升至小时级。

阶段	关键技术	涉及skill
假设检验	H0 / H1假设	ascendc-coding-standards Skill
证据收集与评估	自信值量化(红线违反+40%、一般违反+20%等)	
检视决策判断	决策规则(>60%判定风险)	
更新记忆	检视模块拥有专享记忆模块，实现检视流程强约束	

开发效率对比(效率提升10倍)		
开发模式	人工开发	AI生成
平均耗时	1天	0.5小时

算子代码检视流程



提示词(仅需包含代码路径和检视模式)

测试报告

Ascend C 算子代码检视报告

检视对象: PR #2592 代码差异
检视日期: 2026-03-12
检视模式: 全功能检视
PR链接: https://gitcode.com/cann/ops-transformer/pull/2592/

检视结果摘要

检视类别	HIGH	MEDIUM	LOW	状态
数值运算安全	3	4	2	❌ 需修复
内存与指针安全	2	1	0	❌ 需修复
资源管理	1	2	1	⚠️ 建议修复
输入验证	2	2	1	❌ 需修复
并发安全	1	2	0	❌ 需修复
总计	9	11	4	-

提供代码片段，Agent自动按编码规范条目进行逐级检视并生成检视报告，耗时约15分钟，消耗约1.5w token，较人工检视2小时效率提升约10倍。

CANNBot应用：测试用例UT/ST生成

支持全量算子UT生成，用例生成有效性100%，代码覆盖率80%+，研发效率从天级别提升至小时级。

支持多种类算子ST生成，覆盖值域正交、异常值等5类场景，用例设计完备度100%，研发效率从天级别提升至小时级。

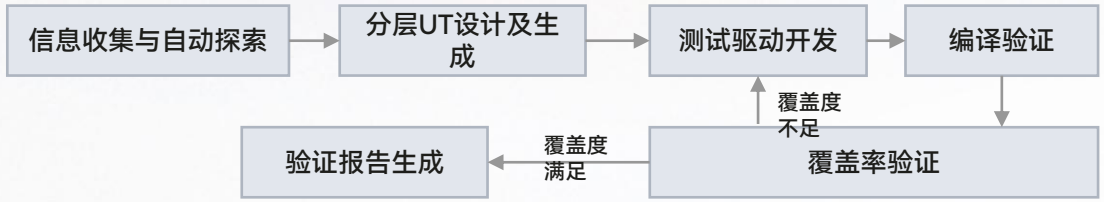
阶段	关键技术	涉及skill
单元测试开发	分层独立测试、测试驱动开发、子Agent协作	ascendc-ut-develop skill
STC测试设计	约束表达模型、拓扑排序约束求解、自动化脚本工具链	ascendc-stc-design skill

开发效率对比(效率提升10+倍)			
开发模式		人工开发	AI生成
平均耗时	UT开发	1天	0.5小时
	ST设计	1天	0.5小时

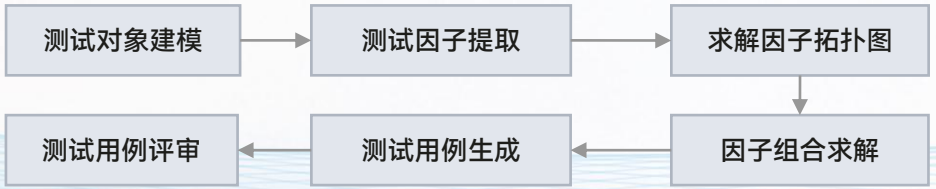
算子批量UT测试结果

算子	算子类别	初始覆盖率	提升后覆盖率	提升幅度	原始UT用例数	新增UT用例数	执行时间(分钟)
moe_distribute_dispatch_v2	mc2	46.50%	68.70%	22.20%	26	16	24
matmul_all_reduce	mc2	65.70%	86.50%	20.80%	22	12	22
moe_distribute_combine_v2	mc2	50.00%	63.60%	13.60%	24	3	24
moe_token_unpermute	moe	82.40%	92.00%	9.60%	27	8	24
moe_update_expert	mc2	77.40%	81.60%	4.20%	12	11	11
grouped_mat_mul_allto_allv	mc2	80.40%	84.50%	4.10%	12	8	36
ffn_to_attention	mc2	88.70%	91.70%	3.00%	13	9	32
matmul_reduce_scatter	mc2	75.90%	78.90%	3.00%	9	5	24

UT生成和执行流程



ST生成流程



1、10+算子，平均25分钟可完成UT生成，平均增加10+用例，有效率100%，代码覆盖率>80%；

2、以Addmm为例算子生成69条ST有效用例，覆盖属性正交，异常值等场景。有效覆盖度95%+。

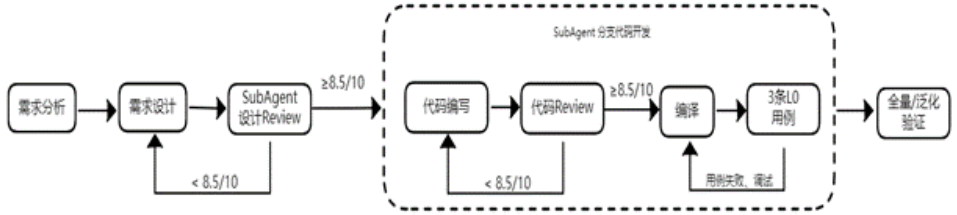
CANNBot应用：算子生成

支持简单/中等难度算子生成能力，复杂场景算子生成陆续上线到开源社区。

阶段	Skill	功能
方案设计	ascendc-api-best-practices	API 使用最佳实践、参数限制
	ascendc-npu-arch	NPU 架构知识、芯片型号映射
	ascendc-docs-search	API 文档索引 + 在线搜索
	ascendc-tiling-design	Tiling 设计方法论，按算子类别分类
代码生成及调试	ascendc-env-check	NPU 设备查询、CANN 环境验证
	ascendc-precision-debug	精度调试，症状-原因速查、常见陷阱
	ascendc-runtime-debug	运行时错误调试，错误码解析、Kernel 挂起排查
	ascendc-ut-develop	UT 开发与覆盖率增强
	ascendc-st-design	aclnn 接口测试用例设计、L0/L1 用例生成
	ascendc-code-review	代码检视方法论、5 大类别规范
	ascendc-task-focus	任务聚焦，解决长任务"迷失在中间"问题

AI驱动算子生成Team

以标准化流程为骨架，以技术工具为抓手，实现Ascend C算子开发全周期、高可靠、可追溯管控。



算子生成效果

性能优化总结

优化成果

1. **核数优化**：小数据量场景核数从 8核降至到2核，降低头开销占比）
2 - **UB内存优化**：移除双缓冲，减少UB内存占用
50%（每队列 16KB → 8KB）
  - 单缓冲大小：16KB(降低 50%)
  - 输入/输出队列大小：64KB(降低 50%)

3. **大数据量扩展性**：支持 1024x4096 测试用例，性能提升明显

  - VEC利用率达到 54-55%
  - SCALAR占比降至 30-35%
  - L2缓存命中率从 0% → 41.7%
  - **MTE2/MTE3带宽利用率提升**：MTE2/MTE3 瀉性从 35.4 摁到 53.0 GB/s) 掣到 12.995GB/s，数据搬运压力
    - **建议**：大缓存命中率在小数据量下较低的原因：
      - 小数据量导致 L2缓存命中率低
      - **建议**：使用更大数据量验证性能扩展性
        - 1024x4096 测试，性能从 2.64us → 11.76us，12.76us → 32.14us
    - **核间负载均衡**：核差异从 13.4% → 0.05us，核间负载非常均衡
  - **建议**：大数据量场景使用全部可用核实现最佳性能
    - 头开销降低到 5-6 微秒

即 大部分算力
2. **L2缓存命中率**(大数据量)：L2缓存作用显著,缓存命中率从 0% → 41.7%，镇小数据量 L2缓存命中率低导致性能问题。
    - **建议**：对于 512KB 的小数据量，保持 8核策略， 但对于极小数据量，增加核数会带来头开销降低和改进核间负载均衡。
同时 性能扩展性也很好（Task Duration降低 50%）。
- 数据量从 128KB 委增至 16MB（1024x4096）
- **精度验证**：所有测试用例通过 ✅
- **性能提升**：
  - 小数据量：Task Duration 11.76us → 2.64us，↓ 4.6x)
  - 大数据量：Task Duration 32.14us → 31.14us
  - VEC利用率：51-55% → 30.97%，↓ 3.1x)
  - SCALAR占比：3.44% → 0.05 us(优化前) / 3.97%

CANN运营日历

——2026年4月活动速递——

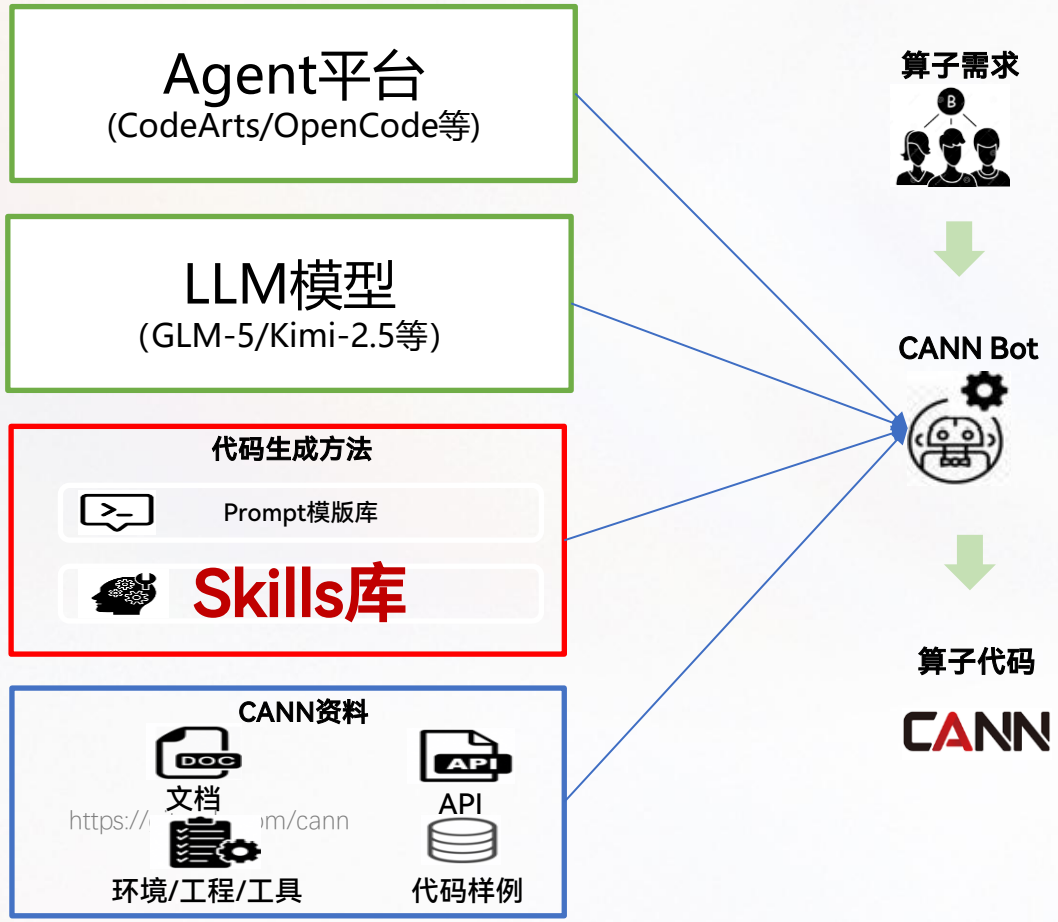
4月运营活动日历

周一	周二	周三	周四	周五	周六
		4月1日	4月2日	4月3日	4月4日
			2026 CANN 训练营 码力全开特辑	CANNBot发布：畅享 算子开发新体验	
4月6日	4月7日	4月8日	4月9日	4月10日	4月11日
			2026 CANN 训练营 码力全开特辑		
4月13日	4月14日	4月15日	4月16日	4月17日	4月18日
CANNBot 学习周-入门课程					
CANNBot入门：从0到1 生成你的第一个算子	CANNBot ASC 算子开发实操	CANNBot PyPTO 算子开发实操	CANNBot TileLang 算子开发实操		★CANN MeetUp 北京站
2026 CANN 训练营 码力全开特辑			2026 CANN 训练营 企业对话专场		
4月20日	4月21日	4月22日	4月23日	4月24日	4月25日
2026 CANN 训练营 码力全开特辑	TileLang典型算子 性能优化		2026 CANN 训练营 企业对话专场		社区TSC线下会议
4月27日	4月28日	4月29日	4月30日		
2026 CANN 训练营 码力全开特辑	从BF16到HiF8：Ascend 950大模型优秀训推案例				

技术直播/沙龙 | 系列课程 | 社区会议 | 高校活动

CANNBot社区建设：CANN社区的AI For AI的辅助研发能力中心

凝聚CANN社区力量，完善AI辅助开发能力，提升算子开发和生态建设效率，同时将端到端的、算子开发最佳实践封装为CANNBot提供给社区。



CANNBot SIG： CANN社区Agent智能体兴趣小组，为社区提供AI辅助开发、算子生成、生态建设的智能体最佳实践

代码仓库： https://gitcode.com/cann/skills

诚挚邀请小伙伴们加入到CANNBot社区，共同完善AI辅助开发能力和提升算子开发效率

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

CANNBot典型特性一览

应用场景

- ASC算子生成
- PyPTO算子生成
- TileLang算子生成
- 模型迁移
- 整网调优
- 资料修复

角色

- 架构Agent
- 开发Agent
- 测试Agent
- 检视Agent

基础能力

- 精度调试
- 性能调试
- UT测试
- ST测试
- 代码检视
- 环境检测
- 资料搜索
- Tiling设计
- VV融合

CANNBot团队正在紧锣密鼓开发更多令人期待的新特性，这些功能将为您带来更加智能、高效和便捷的使用体验